

Lab 07

Threads

Course: Operating System

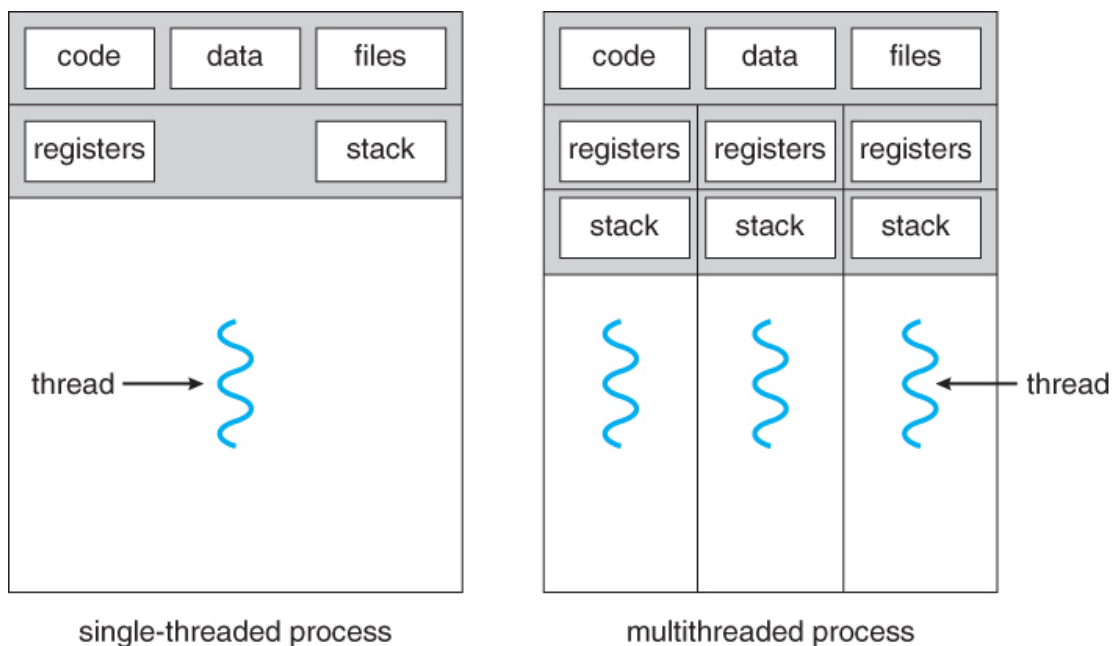
Class: Computer Science IV

Instructor: Bilal Ahmed

Learning Objectives:

- Learn about threading and its applications.
- Creating simple threads.
- Witnessing threads effect.
- Experience Real-life thread examples

Multithreading is indeed a technique that enables concurrent execution of multiple parts of a program, allowing them to run simultaneously on a CPU. By utilizing multiple threads, a program can achieve better CPU utilization and potentially improve overall performance, especially on multi-core processors.



Each thread represents a separate flow of execution within the program, with its own stack, instruction pointer, and program counter. Threads can execute independently of each other, performing different tasks concurrently. This concurrency enables tasks such as handling user input, performing background computations, and responding to events to occur simultaneously, providing a more responsive and efficient application.

Creating a simple thread: We can create threads using pthread library in C as demonstrated in below simple example.

Simple C Code

```
#include <stdio.h>
#include <pthread.h>

void * hello(void *input) {
    printf("hello world\n");
    return NULL;
}

int main(void) {
    pthread_t tid;
    pthread_create(&tid, NULL, hello, NULL);
    pthread_join(tid, NULL);
    return 0;
}
```

Explanation:

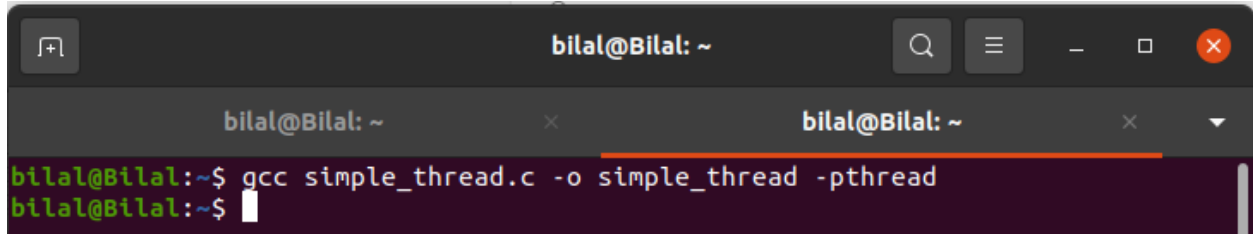
pthread_create(&tid, NULL, hello, NULL);

- This function creates a new thread and starts its execution. It takes four arguments:
- &tid: A pointer to a pthread_t variable where the identifier of the newly created thread will be stored.
- NULL: This argument specifies the thread attributes. Passing NULL indicates that default attributes should be used.
- hello: The function pointer to the function that will be executed by the new thread.
- NULL: An optional parameter that you can use to pass data to the thread. In this case, it's not used, so it's NULL.

pthread_join(tid, NULL);

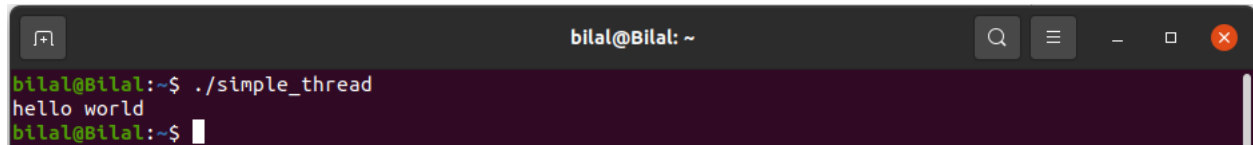
- This function blocks the execution of the calling thread until the specified thread (tid) terminates. It takes two arguments:
- tid: The identifier of the thread to wait for.
- NULL: A pointer to a location where the return value of the thread function can be stored. Since the return value of the thread function is not needed in this case, it's NULL.

Compiling with linking to pthread: Along with compilation we will have to link this code with pthread library to execute it as below.



```
bilal@Bilal: ~  
bilal@Bilal:~$ gcc simple_thread.c -o simple_thread -pthread  
bilal@Bilal:~$
```

Output

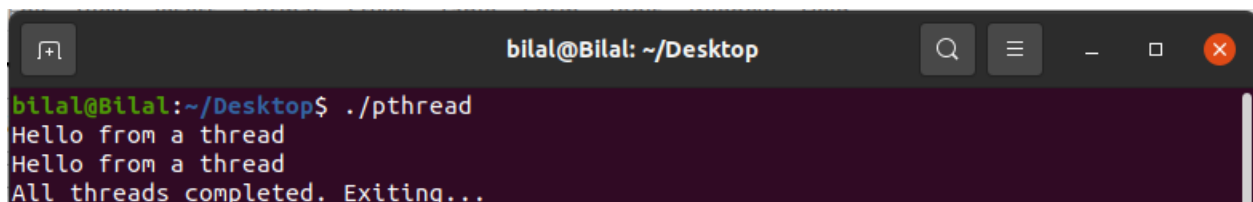


```
bilal@Bilal:~$ ./simple_thread  
hello world  
bilal@Bilal:~$
```

Simple C Code for multiple threads.

```
#include <stdio.h>  
#include <pthread.h>  
  
void *thread_function(void *arg) {  
    printf("Hello from a thread\n");  
    return NULL;  
}  
  
int main() {  
    pthread_t thread1, thread2;  
  
    pthread_create(&thread1, NULL, thread_function, NULL);  
    pthread_create(&thread2, NULL, thread_function, NULL);  
  
    pthread_join(thread1, NULL);  
    pthread_join(thread2, NULL);  
  
    printf("All threads completed. Exiting...\n");  
  
    return 0;  
}
```

Output

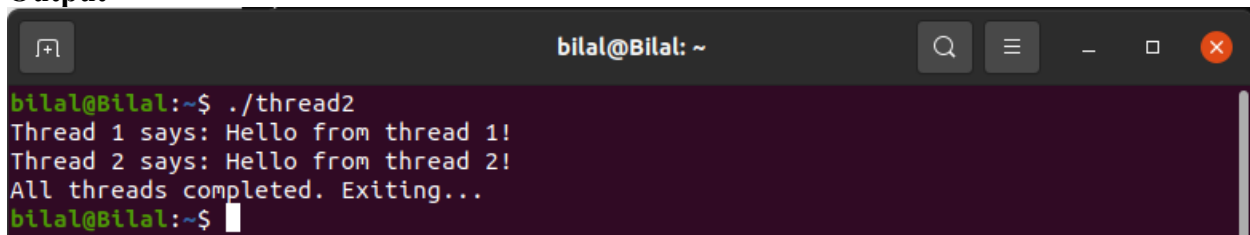


```
bilal@Bilal:~/Desktop$ ./pthread  
Hello from a thread  
Hello from a thread  
All threads completed. Exiting...
```

C Code for Passing data to threads.

```
#include <stdio.h>
#include <pthread.h>
// Thread function that accepts an integer ID and a string message
void *thread_function(void *arg) {
    int thread_id = *(int *)arg; // Cast and dereference the argument
    char *message = (thread_id == 1) ? "Hello from thread 1!" : "Hello from thread 2!";
    printf("Thread %d says: %s\n", thread_id, message);
    return NULL;
}
int main() {
    pthread_t thread1, thread2;
    int id1 = 1, id2 = 2;
    pthread_create(&thread1, NULL, thread_function, (void *)&id1);
    pthread_create(&thread2, NULL, thread_function, (void *)&id2);
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    printf("All threads completed. Exiting...\n");
    return 0;
}
```

Output

A terminal window titled 'bilal@Bilal: ~' with standard window controls. The terminal shows the execution of a C program. The output is as follows:

```
bilal@Bilal:~$ ./thread2
Thread 1 says: Hello from thread 1!
Thread 2 says: Hello from thread 2!
All threads completed. Exiting...
bilal@Bilal:~$
```

Effect of threads.

Code without threads

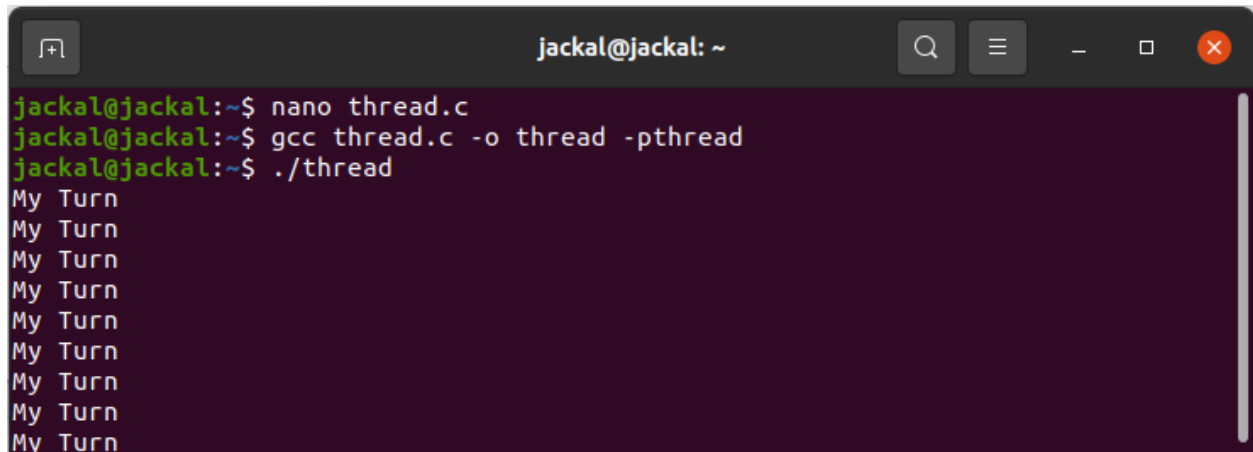
```
#include<unistd.h>
#include<stdio.h>

void myturn() {
    while(1){
        sleep(1);
        printf("My Turn\n");}
}

void yourturn() {
    while(1){
        sleep(1);
        printf("Your Turn\n");}
}

int main(){
    myturn();
    yourturn();
}
```

Output:



```
jackal@jackal: ~
jackal@jackal:~$ nano thread.c
jackal@jackal:~$ gcc thread.c -o thread -pthread
jackal@jackal:~$ ./thread
My Turn
My Turn
My Turn
My Turn
My Turn
My Turn
My Turn
My Turn
My Turn
My Turn
```

Code with threads.

```
#include <pthread.h>
#include<unistd.h>
#include<stdio.h>

void* myturn(void * arg) {
    while(1){
        sleep(1);
```

```

    printf("My Turn\n");
}

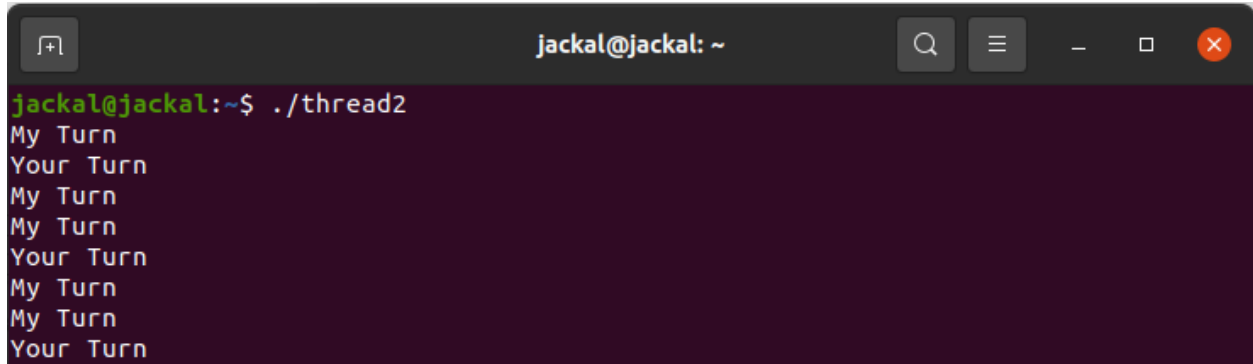
return NULL;
}

void * yourturn(void * arg) {
    while(1){
        sleep(1);
        printf("Your Turn\n");
    }
}

int main(){
    pthread_t newthread1 , newthread2;
    pthread_create(&newthread1, NULL, myturn, NULL);
    pthread_create(&newthread2, NULL, yourturn, NULL);
    pthread_join(newthread1, NULL);
    pthread_join(newthread2, NULL);
}

```

Output:



```

jackal@jackal: ~
jackal@jackal:~$ ./thread2
My Turn
Your Turn
My Turn
My Turn
Your Turn
My Turn
My Turn
Your Turn

```

C code for parallelism.

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <stdint.h> // Include for intptr_t type
#define NUM_THREADS 4
// Function to find the sum of an array segment
void *sum_array_segment(void *arg){
    int *arr = (int *)arg;

```

```

int sum = 0;

// Calculate the sum of the array segment
for (int i = 0; i < 5; i++) {
    sum += arr[i];
}

// Print thread details and sum
printf("Thread %lu: Sum of array segment: %d\n", pthread_self(), sum);

// Cast the sum to void pointer before returning
return (void *)(intptr_t)sum;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int data[NUM_THREADS][5]; // 4 segments of 5 integers each
    int total_sum = 0;

    // Initialize data
    for (int i = 0; i < NUM_THREADS; i++) {
        for (int j = 0; j < 5; j++) {
            data[i][j] = (i * 5) + j + 1; // Populate with consecutive numbers
        }
    }

    // Create threads to compute the sum of each segment
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, sum_array_segment, (void *)data[i]);
    }

    // Join threads and accumulate the total sum
    for (int i = 0; i < NUM_THREADS; i++) {
        void *segment_sum;
        pthread_join(threads[i], &segment_sum);
        total_sum += (int)(intptr_t)segment_sum;
    }

    printf("Total sum of array elements: %d\n", total_sum);

    return 0;
}

```

```
bilal@Bilal: ~  
roscore http://192.168.1... x bilal@Bilal: ~ x bilal@Bilal: ~ x  
bilal@Bilal:~$ ./thread3  
Thread 140652211345152: Sum of array segment: 15  
Thread 140652186167040: Sum of array segment: 90  
Thread 140652202952448: Sum of array segment: 40  
Thread 140652194559744: Sum of array segment: 65  
Total sum of array elements: 210  
bilal@Bilal:~$
```

Exercise

Task 1: Write the C code which should pass data to 3 threads.

Task 2: Write C code which should take two integers from the users in the start and create three threads for addition, multiplication and subtraction.

Task 3: Write C code in which a client-server code in which a thread (server) listens to another thread (client).